



AaDS

МФТИ ВШПИ

Лектор: Денис Аксёненко

Конспект: Денис Войтенко

Оглавление

II семестр	1
Лекция 4. Поиск кратчайших путей в графах	2
4.1 Введение и постановка задачи	2
4.2 Алгоритм Дейкстры	3
4.2.1 Корректность	4
4.2.2 Эвристический подход	7
4.3 Алгоритм Форда-Беллмана	10
4.4 Алгоритм Флойда-Уоршелла	12
Лекция 5. СНМ и миностовы	14
5.1 Введение	14
5.2 Система непересекающихся множеств	14
5.2.1 Лес непересекающихся множеств	15
5.2.1.1 Весовая эвристика	16
5.2.1.2 Ранговая эвристика *	18
5.2.1.3 Эвристика сжатия путей	20
5.2.1.4 Объединение по рангу со сжатием путей	21
5.3 Минимальные остовные деревья	22
5.3.1 Лемма о безопасном ребре	22
5.3.2 Алгоритм Крускала	25
5.3.3 Алгоритм Прима	27

II семестр

Лекция 4

Поиск кратчайших путей в графах

4.1 Введение и постановка задачи

Наверное, одна из самых популярных и полезных задач, которая связана с графами – это нахождение пути из точки A в точку B . В частности такую задачу решает, очевидно, навигатор ведь карта Москвы достаточно просто переносится в графовую интерпретацию и запрос «построй маршрут от работы до дома» сводится к задаче нахождения некоторой последовательности рёбер, по которым мы доезжаем до нашего уютного очага. Формально поставим нашу задачу.

Определение 4.1.1. Пусть дан граф $G\langle V, E \rangle$. Путем из u_a в u_b будем называть конечную последовательность рёбер $P = \{(u_a, v_1), (v_1, v_2) \dots (v_n, u_b)\}$, что тождественно обозначению $P = \{u_a \rightarrow v_1 \rightarrow \dots \rightarrow u_b\}$.

Определение 4.1.2. Пусть $P = \{u_a \rightarrow v_1 \rightarrow \dots u_b\}$ - путь. Тогда $|P| \stackrel{def}{=} n - 1$.

Задача. Пусть дан граф $G\langle V, E \rangle$, а также $u_a, u_b \in V$. Необходимо найти путь из u_a в u_b .

Но такую задачу мы в принципе уже научились решать, просто запуская DFS или BFS. Скорее, как пользователю навигатора, нам очень хотелось бы побыстрее добраться к семье, поэтому мы хотим получить *наикратчайший путь*. Для того чтобы говорить о «наикратчайшести» нужно придать ребрам длину или же задать *весовую функцию*.

Определение 4.1.3. Пусть дан граф $G\langle V, E \rangle$. Весовой функцией будем называть отображение $w : E \rightarrow \mathbb{R}$.

Например, в навигаторе в качестве весовой функции можно считать некоторую комбинацию расстояния и, например, пробок.

Определение 4.1.4. Пусть дан граф $G\langle V, E \rangle$, весовая функция $w : E \rightarrow \mathbb{R}$, а также $u, v \in V$ и некоторый путь $P = \{e_k\}_{k=1}^n$ от u до v . Тогда весом пути примем:

$$w(P) \stackrel{def}{=} \sum_{k=1}^n w(e_k)$$

Определение 4.1.5. Пусть дан граф $G\langle V, E \rangle$, весовая функция $w : E \rightarrow \mathbb{R}$, а также $u, v \in V$. Кратчайшим путём из u в v будем называть путь минимальный по весу и его вес будем обозначать $\delta(u, v)$.

Собственно, вот задача, которую мы будем решать всю лекцию.

Задача. Пусть дан граф $G\langle V, E \rangle$, весовая функция $w : E \rightarrow \mathbb{R}$, а также $u_a, u_b \in V$. Необходимо найти кратчайший путь из u_a в u_b .

Всюду далее в лекции считаем, что задан некоторый граф $G\langle V, E \rangle$ и весовая функция $w : E \rightarrow \mathbb{R}$.

Определение 4.1.6. Множество соседей вершины u в графе будем обозначать $\alpha(u)$.

4.2 Алгоритм Дейкстры

Первый алгоритм, который мы рассмотрим – это алгоритм Дейкстры. Он позволяет вычислять кратчайшие расстояния от $s \in V$ до всех вершин в

графе, если у нас неотрицательная весовая функция. Далее считаем, что *весовая функция неотрицательна*.

Algorithm 1 Алгоритм Дейкстры

```

1: function DIJKSTRA( $G, w, s$ )
2:    $\delta_c(s, v) = +\infty, \forall v \in V$ 
3:    $\delta_c(s, s) = 0$ 
4:    $K = \{s\}$   $\triangleright$  Множество вершин, до которых мы знаем расстояние.
5:   while  $K \neq V$  do
6:      $u \leftarrow \underset{v \in V \setminus K}{\operatorname{argmin}} \delta_c(s, v)$   $\triangleright$  Получаем вершину из неизвестных с
        минимальным  $\delta$ 
7:      $K \leftarrow K \cup \{u\}$   $\triangleright$  Утверждается, что для  $u$   $\delta$  посчитана
        корректна (док-во см. далее)
8:     for  $v \in \alpha(u)$  do
9:        $\delta_c(s, v) \leftarrow \min\{\delta_c(s, v), \delta_c(s, u) + w(u, v)\}$ 
10:  return  $\delta_c$ 

```

Определение 4.2.1. Обновление δ_c в строчках 8-9 будем называть *релаксацией*.

4.2.1 Корректность

Далее приведем доказательство корректности, но перед этим докажем мини-лемму. Далее под δ_c будем понимать «расстояния» из алгоритма выше. В последствии мы покажем, что $\delta_c \equiv \delta$.

Лемма 4.2.1. $\forall v \in V \hookrightarrow \delta(s, v) \leq \delta_c(s, v)$

Доказательство. В силу 9-ой строчки алгоритма $\exists v_1 \in V : \delta_c(s, v_0) = \delta_c(s, v_1) + w(v_1, v_0)$ (то есть раньше была какая-то вершина, из которой мы произвели релаксацию v_0). Но также мы можем сказать и про v_1 , действительно, $\exists v_2 \in V : \delta_c(s, v_1) = \delta_c(s, v_2) + w(v_2, v_1)$. Итого:

$$\delta_c(s, v_0) = \delta_c(s, v_2) + w(v_2, v_1) + w(v_1, v_0)$$

Будем применять последовательно рассуждения выше к δ_c , пока не дойдем

до s , и получим, что:

$$\delta_c(s, v_0) = \sum_{k=1}^m w(v_k, v_{k-1}), v_m = s$$

Значит, мы нашли путь $P_0 = \{s \rightarrow v_{m-1} \rightarrow v_{m-2} \rightarrow \dots \rightarrow v_0\}$ с весом $\delta_c(s, v_0)$. Далее воспользуемся определением δ и получим требуемое:

$$\delta(s, v_0) \stackrel{\text{def}}{=} \min_P w(P) \leq w(P_0) = \delta_c(s, v_0)$$

$$\delta(s, v_0) \leq \delta_c(s, v_0)$$

□

Теорема 4.2.2. $\delta_c \equiv \delta$, то есть получившиеся на выходе расстояния корректно посчитаны.

Доказательство.

Покажем по индукции, что утверждение верно. Начнем с базы, а именно с начальной вершины. Действительно, $\delta_c(s, s) = 0 = \delta(s, s)$.

Пусть на 6-ой строчке алгоритма мы достаём вершину t . Предположим $\delta(s, t) \neq \delta_c(s, t)$, т.е. $\exists P$ - кратчайший путь, т.ч. $w(P) < \delta_c(s, t)$. На этом пути возьмем самую первую вершину v такую, что $v \in V \setminus K$. Далее возьмем ей предшествующую u . Ясно, что $u \in K$, по определению v .

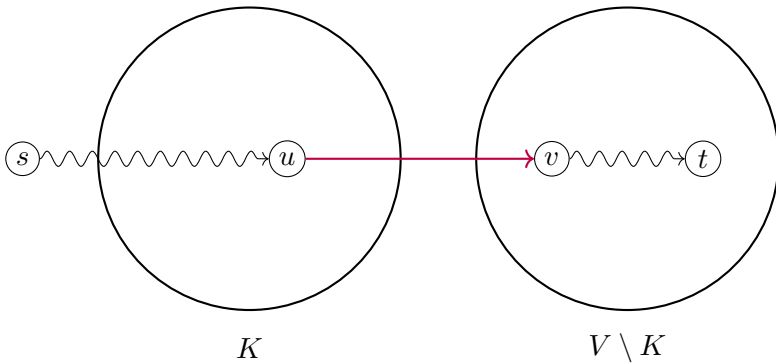


Рис. 4.1: Предположение, что существует другой более короткий путь

В силу кратчайшести P , кратчайшим является, в частности, путь $s \rightsquigarrow v$. А значит:

$$\delta(s, v) = \delta(s, u) + w(u, v)$$

Комбинируя равенство выше, 9-ую строчку алгоритма и $\delta_c(s, u) = \delta(s, u)$ (см. предположение):

$$\delta_c(s, v) \leq \underbrace{\delta_c(s, u)}_{\delta(s, u)} + w(u, v) = \delta(s, u) + w(u, v) = \delta(s, v)$$

Значит:

$$\delta_c(s, v) \leq \delta(s, v)$$

Но по вышедоказанной лемме:

$$\delta(s, v) \leq \delta_c(s, v) \leq \delta(s, v) \Rightarrow \delta(s, v) = \delta_c(s, v)$$

Далее, коль скоро мы достаём вершину t , значит, у неё минимальное δ_c из всех, а значит $\delta_c(s, t) \leq \delta_c(s, v)$. Учитывая вышедоказанное, верно:

$$\delta_c(s, t) \leq \delta_c(s, v) = \delta(s, v)$$

Но вспомним, что v стоит раньше t в кратчайшем пути из s в t , значит $\delta(s, v) \leq \delta(s, t)$ (см. картинку для наглядности). Значит:

$$\delta_c(s, t) \leq \delta(s, v) \leq \delta(s, t)$$

$$\delta_c(s, t) \leq \delta(s, t)$$

Но снова применяя вышедоказанную лемму, получаем требуемое противоречие с тем, что $\delta(s, t) \neq \delta_c(s, t)$:

$$\delta(s, t) \leq \delta_c(s, t) \leq \delta(s, t) \Rightarrow \delta(s, t) = \delta_c(s, t)$$

□

Теперь рассмотрим время работы алгоритма. Заметим, что каждую вершину мы берём ровно один раз (время выбора зависит от реализации, обозначим его пока за $\tau(n)$). Далее, каждое ребро мы тоже смотрим ровно один раз и обновляем в цикле релаксации вершину за время $\eta(n)$. Значит, общее время работы: $\Theta(|V|\tau(|V|) + |E|\eta(|V|))$.

Рассмотрим теперь стратегии выбора вершины с минимальным δ_c :

1. Наивно. Просто ищем минимум в массиве всех вершин за $\Theta(|V|)$, а обновление в цикле релаксации – это $\Theta(1)$. Тогда итоговая асимптотика $\Theta(|V|^2 + |E|)$.
2. Двоичная куча. Поиск минимума – $\mathcal{O}(\log |V|)$, обновление тоже $\mathcal{O}(\log |V|)$. Итоговая асимптотика $\mathcal{O}(|V| \log |V| + |E| \log |V|)$. Но, вспомним, что $|V| = \mathcal{O}(|E|)$, поэтому $\mathcal{O}(|V| \log |V| + |E| \log |V|) = \mathcal{O}(|E| \log |V|)$. Итого $\mathcal{O}(|E| \log |V|)$.

Примечание. Вообще говоря, асимптотика еще зависит от способа хранения графа, кардинально они не меняются, но, например, если мы реализовали хранение графа в виде хеш-таблицы, то асимптотики будут уже *в среднем* смысле.

Пусть теперь мы хотим найти кратчайшее расстояние от вершины s до некоторой вершины t . Следующее следствие из теоремы даёт способ расчета $\delta(s, t)$.

Следствие 4.2.3. Пусть $t \in V$. Тогда чтобы найти $\delta(s, t)$, достаточно завершить алгоритм Дейкстры в момент, когда на 6-ой строчке была получена вершина t .

4.2.2 Эвристический подход

Давайте немного подробнее рассмотрим алгоритм Дейкстры и улучшим решение задачи нахождения $\delta(s, t)$. Если взять алгоритм как есть, то при достаточно плохом графе «зона поиска» будет достаточно большой.

И напрашивается такая мысль, что быть может не всегда оптимально так наивно перебирать вершины, и использовать какие-то общие приземленные данные о задаче и как-то это отразить в весовой функции, чтобы приотизировать выбор некоторых рёбер. Такие соображения о задаче называют *эвристическими*.

Определение 4.2.2. Функцию $h : V^2 \rightarrow \mathbb{R}^+$ будем называть эвристикой.

Теперь интегрируем нашу эвристику в алгоритм Дейкстры (такой алгоритм называется A^*):

Algorithm 2 Алгоритм A^*

```

1: function ASTAR( $G, w, s, t, h$ )
2:    $\delta_c(s, v) = +\infty, \forall v \in V$ 
3:    $\delta_c(s, s) = 0$ 
4:    $K = \{s\}$ 
5:   while  $K \neq V$  do
6:      $u \leftarrow \operatorname{argmin}_{v \in V \setminus K} [\delta_c(s, v) + h(v, t)]$   $\triangleright$  Заметьте, что здесь используется
        эвристика
7:     if  $u = t$  then
8:       return  $\delta_c(s, u)$ 
9:      $K \leftarrow K \cup \{u\}$ 
10:    for  $v \in \alpha(u)$  do
11:       $\delta_c(s, v) \leftarrow \min\{\delta_c(s, v), \delta_c(s, u) + w(u, v)\}$ 
12:  return  $\delta_c$ 

```

Теперь встаёт вопрос о том какие вообще эвристические функции мы можем брать, ведь если взять достаточно плохую функцию h , то можно заставить алгоритм A^* работать дольше Дейкстры или вообще сходиться к неверному ответу.

Определение 4.2.3. Пусть $t \in V$. Эвристику h будем называть *допустимой*, если $\forall v \in V \hookrightarrow h(v, t) \leq \delta(v, t)$.

Примечание. Допустимость эвристики напрямую зависит от пункта назначения.

Далее приведём базовое утверждение, которое на экзамене будет без доказательства (но для интересующегося темой читателя оно приведено).

Теорема 4.2.4 (б/д). Если эвристика допустима, то A^* вернет верный результат, т.е. найдет оптимальный путь.

Доказательство. Предположим противное. Пусть A^* вернул неверный результат δ^* . Ясно, что $\delta(s, t) < \delta^*$. Заметим, что доколе алгоритм работает, то в K обязана находиться хотя бы одна вершина v , которая находится на оптимальном пути до t . Пусть это не так, значит в частности нет и t , ведь, очевидно, t лежит на таком пути. Коль скоро v лежит на оптимальном пути, $\delta(s, t) = \delta(s, v) + \delta(v, t)$ (предлагается это читателю

доказать, как простое логическое упражнение). В силу допустимости эвристики заключаем:

$$\delta(s, v) + h(v, t) \leq \delta(s, v) + \delta(v, t) = \delta(s, t)$$

$$\delta_c(s, v) + h(v, t) \leq \delta(s, v) + h(v, t) \leq \delta(s, t)$$

$$\delta_c(s, v) + h(v, t) \leq \delta(s, t)$$

Заметим, что $h(t, t) \leq \delta(t, t) = 0 \Rightarrow h(t, t) = 0 \Rightarrow \delta_c(s, t) = \delta_c(s, t) + h(t, t)$. По вышедоказанной лемме:

$$\delta(s, t) \leq \delta_c(s, t) = \delta_c(s, t) + h(t, t)$$

Далее:

$$\delta_c(s, v) + h(v, t) \leq \delta(s, t) \leq \delta_c(s, t) + h(t, t)$$

$$\delta_c(s, v) + h(v, t) \leq \delta_c(s, t) + h(t, t)$$

Значит, мы не вытащим вершину t , пока не вытащим из K все вершины находящиеся на оптимальном пути (см. определение вершины v), а найденное $\delta_c(s, t)$ будет валидно посчитано. $\square$

Определение 4.2.4. Эвристику h будем называть монотонной, если $\forall (u, v) \in E \hookrightarrow h(u, t) - h(v, t) \leq w(u, v)$ и $h(t, t) = 0$.

Утверждение 4.2.5. Всякая монотонная эвристика допустима.

Доказательство. Покажем корректность утверждения по индукции по рёберной длине.

База. $0 = h(t, t) \leq \delta(t, t) = 0$.

Переход. Пусть показали для длины $n_0 - 1$, покажем для n_0 . Возьмем вершину t такой что до неё расстояние от s равно n_0 . Тогда возьмем вершину на кратчайшем пути перед t на расстоянии $n_0 - 1$ рёбер – v (то есть из вершины s до v есть ребро). Значит для вершины v можем применить предположение индукции: $h(v, t) \leq \delta(v, t)$. Комбинируя с тем, что $h(s, t) - h(v, t) \leq w(s, v)$ получаем требуемое:

$$h(s, t) \leq w(s, v) + h(v, t) \leq w(s, v) + \delta(v, t) = \delta(s, t)$$

$\square$

Теорема 4.2.6 (б/д). Если эвристика монотонна, то алгоритм A^* выдает корректный результат и работает по времени не медленнее, чем алгоритм Дейкстры.

4.3 Алгоритм Форда-Беллмана

Теперь давайте решим ту же самую задачу нахождения расстояния от s до всех остальных с помощью динамического программирования – алгоритма Форда-Беллмана. В отличие от алгоритма Дейкстры этот алгоритм расширяет образ весовой функции до всей числовой прямой. Однако необходимо требовать, чтобы в графе не было циклов отрицательного веса. Заведём двумерную динамику $d[k][v]$ – длина кратчайшего пути из s в t , состоящего из k рёбер. Тогда легко видеть, что формула перехода представляет собой следующее:

$$d[k][v] = \min_{(u,v) \in E} [d[k-1][u] + w(u, v)]$$

То есть мы должны на каждом шаге динамики «расширять» путь до v из смежных к ней вершин. База динамики очевидна: $dp[0][s] = 0, dp[0][v] = +\infty \forall v \in V \setminus \{s\}$. А ответ тогда вычисляется как: $\delta(s, t) = \min_{k \in 0 \dots |V|-1} dp[k][t]$.

Algorithm 3 Алгоритм Форда-Беллмана

```

1: function FORD_BELLMAN( $G, w, s$ )
2:    $dp[0][s] \leftarrow 0$ 
3:    $dp[0][v] \leftarrow +\infty \forall v \in V \setminus \{s\}$ 
4:   for  $k \in 1 \dots |V| - 1$  do
5:     for  $(u, v) \in E$  do
6:        $dp[k][v] \leftarrow \min(dp[k][v], dp[k-1][u] + w(u, v))$ 
7:   return  $dp$ 
```

Время работы данного алгоритма $\Theta(|V||E|)$.

Что если в вышеописанном алгоритме мы сделаем еще одну итерацию ($|V|$ -ую)? А точнее, что если до какой-то вершины уменьшится расстояние в dp ? Коль скоро мы рассматривали уже пути длины $|V|$, то эти пути точно

самопересекаются (ведь максимальная длина несамопересекающегося пути – это ровно $|V| - 1$). Отсюда напрашивается важное утверждение.

Теорема 4.3.1. $\exists v \in V : \min_{k \in 0 \dots |V|-1} dp[k][v] < dp[v][|V|] \iff$ существует цикл отрицательного веса, достижимый из s .

Доказательство. Пусть $\exists v \in V : \min_{k \in 0 \dots |V|-1} dp[v][k] < dp[v][|V|]$. Значит $\exists P = \{(s, u_1), (u_1, u_2) \dots (u_n, v)\}$ – самопересекающийся путь $\Rightarrow$ хотя бы один цикл существует. Предположим, что все циклы неотрицательны.

В силу того, что расстояние уменьшилось на $|V|$ -ой итерации, то $w(P) < \min_{k \in 0 \dots |V|-1} dp[k][v]$. Далее, на основе пути P построим путь P' . Приведём алгоритм построения P' , пусть U – множество раскрытых вершин (изначально пустое) и k – номер рассматриваемого ребра (изначально $k = 0$, то есть рассматриваем (s, u_1)).

1. Если $k > n$, то алгоритм считаем оконченным. Рассмотрим k -ое ребро (u_f, u_t) . Если $u_t = v$, то добавляем (u_f, u_t) в конец P' и алгоритм завершаем. Если $u_t \in U$, то перейти к шагу 2, иначе 3.
2. Удаляем с конца P' ребра (u_a, u_b) , пока $u_b \neq u_t$, где u_t – вершина из пункта 1. После инкрементируем k и переходим к шагу 1.
3. Добавляем конец P' ребро (u_f, u_t) и в множество U вершину u_t , инкрементируем k и переходим к шагу 1.

По сути алгоритм строит P' и как только он видит, что путь заиклился, он удаляет цикл удалением рёбер из пути. Далее рассмотрим важные свойства полученного пути:

- P' – путь от s до v .
- P' – простой. Действительно, в силу того, что мы поддерживаем множество раскрытых вершин, то все вершины в пути различны.
- $|P'| \leq |V| - 1$, как следствие простоты пути.
- $w(P') \leq w(P)$. В силу того, что мы удаляем циклы из пути P , которые являются неотрицательными, то легко видеть, что вес пути P' уж точно не превосходит вес P .

Далее, $|P'| \leq |V| - 1 \Rightarrow \exists k' < |V| - 1 : dp[k'][v] \leq w(P')$. Теперь заметим следующее:

$$\min_{k \in 0 \dots |V|-1} dp[k][v] \leq dp[k'][v] \leq w(P') \leq w(P) < \min_{k \in 0 \dots |V|-1} dp[k][v]$$

$$\min_{k \in 0 \dots |V|-1} dp[k][v] < \min_{k \in 0 \dots |V|-1} dp[k][v]$$

Получили противоречие.

В другую сторону, пусть существует цикл отрицательного веса $C = \{c_1, \dots, c_k\}$, $k \leq |V| - 1$ достижимый из s . Значит на $|V|$ -й итерации есть вершина цикла, которую мы рассмотрим второй раз вдоль цикла, а значит мы обновим минимум. $\square$

4.4 Алгоритм Флойда-Уоршелла

Всюду далее мы опять считаем, что образ весовой функции вещественен (не только положительный). Теперь наша задача состоит в том, чтобы найти расстояние между всеми парами вершин.

Давайте пронумеруем все вершины $V = \{v_1, \dots, v_{|V|}\}$ и заведём динамику $dp[k][u][v]$ – минимальный вес пути из u в v по рёбрам с номерами не больших k . Далее воспользуемся следующей идеей:

$$\delta(u, v) = \min_{p \in V} [\delta(u, p) + \delta(p, v)]$$

Будем делать переход $dp[k][u][v] = \min(dp[k-1][u][v_k] + \delta[k-1][v_k][v], dp[k][u][v])$, по сути пытаясь выбрать оптимальную «среднюю вершину».

Algorithm 4 Алгоритм Флойда-Уоршелла

```

1: function FLOYD_WARSHALL( $G, w$ )
2:    $dp[0][u][u] \leftarrow 0, \forall u \in V$ 
3:    $dp[k][u][v] = +\infty, \forall k \in [1, n], \forall u \neq v \in V$ 
4:   for  $k \in 1 \dots |V|$  do
5:     for  $u \in V$  do
6:       for  $v \in V$  do
7:          $dp[k][u][v] \leftarrow \min(dp[k-1][u][v_k] + dp[k-1][v_k][v], dp[k-1][u][v])$ 
8:   return  $dp$ 

```

Очевидно, ответ будет в $dp[|V|][u][v]$. Асимптотика $\Theta(|V|^3)$.

Аналогично алгоритму Форда-Беллмана здесь тоже можно искать циклы отрицательного веса.

Теорема 4.4.1. В графе есть циклы отрицательного веса $\iff \exists v \in V : dp[|V|][v][v] < 0$

Доказательство. $\exists v \in V : dp[|V|][v][v] < 0 \iff \exists k \in 1 \dots |V| : dp[|V|][v][v] = dp[|V|-1][v][v_k] + dp[|V|-1][v_k][v] < 0 \iff$ В графе есть цикл отрицательного веса.

Пояснение: во второй эквивалентности мы по сути утверждаем, что существует такая вершина v_k , что путь из v в v (то есть цикл) будем отрицательного веса, поэтому третья эквивалентность верна. $\square$

Лекция 5

СНМ и миностовы

5.1 Введение

На этой лекции мы продолжим работать со взвешенными структурами данных. На ней мы решим задачу нахождения *минимальных остовных деревьев*, а также смежную структуру данных – *систему непересекающихся множеств*, которая нам позволит быстрее выполнять поиск искомого дерева.

Всюду далее у нас есть граф $G\langle V, E \rangle$ и на множестве рёбер задана весовая функция $w : E \rightarrow \mathbb{R}$.

5.2 Система непересекающихся множеств

Определение 5.2.1. Системой непересекающихся множеств назовём структуру данных, которая в любой момент поддерживает некоторый набор множеств $\mathcal{A} = \{A_1, A_2, \dots, A_n\} : i \neq j \Rightarrow A_i \cap A_j = \emptyset$, а также позволяет выполнять три операции:

- **MAKE_SET**(x) – создает множество с единственным элементом x , который не принадлежит ни одному множеству.

- $\text{SAME}(x, y)$ – возвращает **true**, если $\exists i : x, y \in A_i$ (то есть два элемента лежат в одном и том же множестве).
- $\text{UNITE}(x, y)$ – выполняет объединение двух множеств, в которых лежат x и y .

Элементы множества A_i иногда называют *представителями множества*.

Упражнение. Предложите алгоритм, который выделяет компоненты связности, с помощью СНМ.

5.2.1 Лес непересекающихся множеств

Далее рассмотрим одну из реализаций СНМ – *лес непересекающихся множеств*.

Каждое множество будем представлять в виде дерева, которое состоит из элементов этого множества.

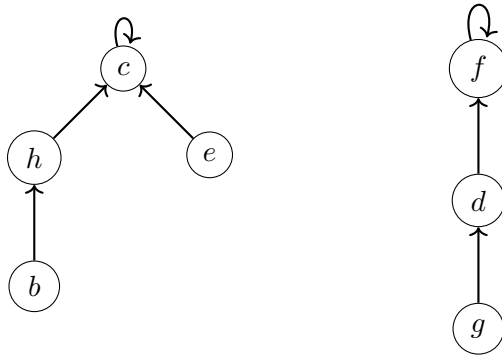


Рис. 5.1: Пример СНМ для $\mathcal{A} = \{\{c, h, b, e\}, \{f, d, g\}\}$

Тогда в такой парадигме интерфейс реализуется следующим образом:

- $\text{MAKE_SET}(x)$ – создание узла.
- $\text{SAME}(x, y)$ – для x и y находим корни деревьев, в которых они лежат,

r_x и r_y при помощи поднятия вверх. Затем сравниваем эти корни, если x и y лежат в одном множестве, то $r_x = r_y$, иначе $r_x \neq r_y$.

- **UNITE**(x, y) – также как в **SAME** находим корни r_x и r_y , а затем подвешиваем r_y в качестве ребенка к r_x .

Итак, исследуем время работы нашей структуры данных. Очевидно, **MAKE_SET** работает за $\mathcal{O}(1)$. Но вот с **UNIT** и **SAME** уже не всё так просто, потому что время их работы зависит напрямую от высот деревьев, в которых лежат x и y . И не сложно заметить, если взять два бамбука высоты h и вызвать **SAME** от самых глубоких вершин, то время работы будет $\Theta(h)$.

На самом деле можно ускорить работу этих операций вплоть до «почти константы». Для этого надо использовать две эвристики: *ранговую* и *сжатие путей*.

5.2.1.1 Весовая эвристика

Кажется достаточно разумным, чтобы уменьшить итоговую высоту поднятия, нужно подвешивать множество меньшего размера к большему. Такая эвристика называется *весовой*. Нам необходимо поддерживать размер текущего множества, что достаточно тривиально:

1. При **MAKE_SET** будет создавать новый узел и в счётчик ставить 1.
2. При **UNITE** будем просто прибавлять к счётчику размера большего дерева счётчик меньшего.

Определение 5.2.2. Пусть x – элемент некоторого множества в СНМ. Тогда $s(x)$, по определению, положим размером данного множества, в котором лежит x .

Определение 5.2.3. Пусть x и y – элементы некоторого множества в СНМ. Тогда $x \cup y$, по определению, результат объединения множеств, в которых они лежат. $s(x \cup y)$ – размер результирующего множества.

Упражнение. Пусть даны два множества в СНМ, первое больше второго. Верно ли утверждение, что высота первого больше высоты второго?

Теорема 5.2.1. Пусть выполнялось m операций `MAKE_SET`, `SAME`, `UNITE`, из которых n операций `MAKE_SET`. Тогда суммарное время работы при использовании весовой эвристики равно $\mathcal{O}(m \log n)$.

Доказательство. Вспомним, что `MAKE_SET` работает за $\mathcal{O}(1)$, поэтому его суммарное время выполнения есть $\mathcal{O}(n)$. В тоже время время основной вклад в асимптотику дают `SAME`, `UNITE`, т.к. они напрямую зависят от того на какой глубине лежат рассматриваемые элементы x и y , ведь мы «прыгаем» от них к корню. Можно сказать, что каждая из них работает за $\Theta(h(x) + h(y))$, где $h(v)$ – глубина, на которой лежит вершина v . Далее приведём оценку h .

Давайте рассмотрим произвольный элемент x и исследуем его положение в течение всей последовательности операций. Изначально, когда только вызвали `MAKE_SET`, он находился на глубине $h(x) = 0$. Далее рассмотрим случаи объединения с некоторым множеством, которое содержит некоторый элемент y :

1. $s(x) > s(y) \Rightarrow$ Тогда мы подвешиваем множество с элементом y к множеству с элементом x и $h(x)$ не меняется.
2. $s(x) < s(y) \Rightarrow$ Тогда мы подвешиваем множество с элементом x к множеству с элементом y . Но в этом случае глубина x увеличилась на единицу. Также заметим, что $s(x \cup y) > 2s(x)$. Действительно, т.к. мы подвесили множество с элементом x к множеству с элементом y , которое не менее чем первое, то мы увеличились минимум в два раза. Более формально:

$$s(x \cup y) = s(x) + s(y) > s(x) + s(x) = 2s(x)$$

3. $s(x) = s(y) \Rightarrow$ Предположим худший случай и мы подвесили множество с x к множеству с y (хотя можно было и наоборот, но опять же смотрим худший случай). Тогда высота x опять увеличилась на единицу и размер результирующего увеличился в два раза:

$$s(x \cup y) = s(x) + s(y) = s(x) + s(x) = 2s(x)$$

Из вышеописанного следует, что высота x увеличивается в худшем случае на единицу. И она увеличивается, тогда и только тогда, когда это приводит

к увеличению множества, в котором лежит x , минимум в два раза. Значит высота x оценивается сверху количеством увечений его множества в два раза. А в наибольшем множестве, котором он мог находиться – это в множестве размера n . Количество увечений минимум в два раза до размера n есть $\mathcal{O}(\log n)$. Значит, высота не более чем логарифмична.

В частности из этого следует, что время работы SAME и UNITE есть $\mathcal{O}(\log n)$. Следовательно, в силу того, что у нас всего m операций, то итоговая асимптотика есть $\mathcal{O}(m \log n)$. $\square$

Следствие 5.2.2. Пусть выполнялось m операций MAKE_SET, SAME, UNITE, из которых n операций MAKE_SET. Тогда амортизированная стоимость операций при использовании весовой есть $\mathcal{O}^*(\log n)$.

Доказательство.

$$\frac{\mathcal{O}(m \log n)}{m} = \mathcal{O}\left(\frac{m \log n}{m}\right) = \mathcal{O}^*(\log n)$$

$\square$

5.2.1.2 Ранговая эвристика *

Есть еще один способ аналогичный прошлому, его иногда используют как альтернативу. Чтобы реже приходиться к ситуации бамбука и уменьшать высоту, надо подвешивать дерево с меньшей высотой к большему, потому что тогда высота почти не меняется. Такой подход называется *ранговой эвристикой*. Будем поддерживать ранг – грубую оценку на высоту дерева. Ранг множества, в котором лежит вершина x , будем обозначать $r(x)$. Считается она следующим образом:

1. При MAKE_SET(x) выставяем множеству ранг 0.
2. При UNITE(x, y) рассмотрим случаи:
 - (а) $r(x) > r(y) \Rightarrow$ подвешиваем дерево, в котором лежит y , к дереву в котором лежит x . Ранг после этого равен $r(x)$ (ведь «высота дерева» не поменялась).

- (b) $r(x) < r(y) \Rightarrow$ подвешиваем дерево, в котором лежит x , к дереву в котором лежит y . Ранг после этого равен $r(y)$ (ведь «высота дерева» не поменялась).
- (c) $r(x) = r(y)$. Тут без разницы к кому что подвешивать, ведь ранг одинаков. Но после подвешивания ранг новоиспеченного множества будет равен $r(x) + 1$ («высота дерева» увеличилась на единицу).

Упражнение. Верно ли что если ранг одного дерева больше другого, то тоже верно для размера соответствующих множеств?

Упражнение. Покажите, что если использовать только ранговую эвристику, то ранг есть максимальная глубина дерева.

Сама по себе эта эвристика дает уже прирост к работе. Следующие два утверждения даны для интересующегося темой читателя и *не будут спрашиваться на экзамене*.

Теорема 5.2.3 (б/д). Пусть выполнялось m операций MAKE_SET, SAME, UNITE, из которых n операций MAKE_SET. Тогда суммарное время работы при использовании ранговой эвристики равно $\mathcal{O}(m \log n)$.

Следствие 5.2.4 (б/д). Пусть выполнялось m операций MAKE_SET, SAME, UNITE, из которых n операций MAKE_SET. Тогда амортизированная стоимость операций MAKE_SET, SAME, UNITE есть $\mathcal{O}^*(\log n)$.

Доказательство.

$$\frac{\mathcal{O}(m \log n)}{m} = \mathcal{O}\left(\frac{m \log n}{m}\right) = \mathcal{O}^*(\log n)$$

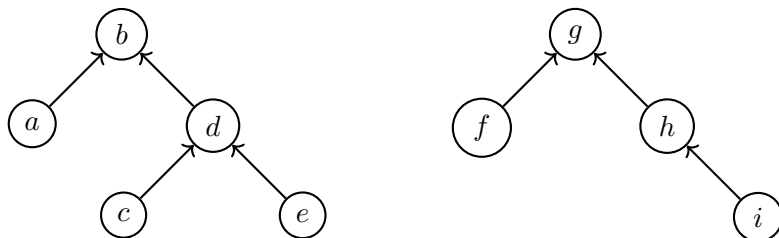
□

То есть получается, уже сама по себе ранговая эвристика ускоряет нас с линейного до в каком-то плане логарифмического времени работы.

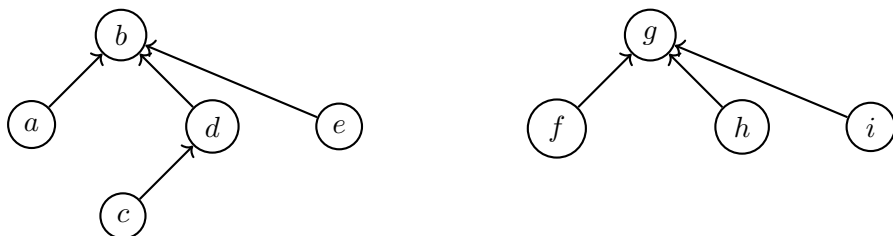
5.2.1.3 Эвристика сжатия путей

Также, кажется что неплохой идеей будет узлы, для которых мы узнали корень, *переподвешивать сразу к корню*.

Рассмотрим на примере работу данной эвристики. Пусть даны два дерева.



И мы хотим проверить что e и i лежат в одном множестве. Тогда после вызова `SAME( $e, i$ )` конфигурация деревьев будет следующая:



Тогда следующий вызов `SAME( $e, i$ )`, очевидно, будет быстрее, ведь нам нужно подняться на один уровень вместо 2-х. Также как и ранговая, сама по себе эвристика сжатия путей тоже дает выигрыш. Также как и с ранговой эвристикой, следующее утверждение на экзамене спрашиваться не будет.

Теорема 5.2.5 (б/д). Если было выполнено n операций `MAKE_SET`, а также m операций `SAME`. Тогда суммарное время работы при использовании ранговой эвристики равно $\mathcal{O}\left(n + \frac{m}{2} \log_{2+\frac{m}{2n}} n\right)$.

5.2.1.4 Объединение по рангу со сжатием путей

Самый большой прирост даёт использование этих двух эвристик одновременно. Если раньше мы могли амортизированно дойти до логарифма, то с двумя эвристиками мы можем дойти до почти константы. Давайте же рассмотрим эту «почти константу».

Определение 5.2.4. Функцией Аккермана назовём следующую функцию:

$$A(m, n) \stackrel{\text{def}}{=} \begin{cases} n + 1, m = 0 \\ A(m - 1, 1), m > 0, n = 0 \\ A(m - 1, A(m, n - 1)), m > 0, n > 0 \end{cases}$$

Функция A является чрезвычайно быстро растущей функцией.

Определение 5.2.5. Обратной функцией Аккермана положим $\alpha(n) \stackrel{\text{def}}{=} \min_{k \in \mathbb{N}} \{k : A(k, k) \geq n\}$.

На практике $\alpha(n) \leq 4$, поэтому в рамках разумных задач, можно считать её константой (но теоретически, все же это функция).

Следующая теорема даёт нам желанный результат. Сие доказательство столь сложно, что Кормен расположил его на 10 страницах А4, поэтому, очевидно, мы принимаем его без доказательства.

Теорема 5.2.6 (б/д). Пусть выполнялось m операций `MAKE_SET`, `SAME`, `UNITE`, из которых n операций `MAKE_SET`. Тогда при использовании обоих эвристик время выполнения есть $\mathcal{O}(m\alpha(n))$.

Следствие 5.2.7 (б/д). Пусть выполнялось m операций `MAKE_SET`, `SAME`, `UNITE`, из которых n операций `MAKE_SET`. Тогда амортизированная стоимость операций есть $\mathcal{O}^*(\alpha(n))$.

Доказательство.

$$\frac{\mathcal{O}(m\alpha(n))}{m} = \mathcal{O}\left(\frac{m\alpha(n)}{m}\right) = \mathcal{O}^*(\alpha(n))$$

□

5.3 Минимальные остовные деревья

Определение 5.3.1. Пусть $G\langle V, E \rangle$ – граф. Тогда граф $G'\langle V', E' \rangle$, будем называть *подграфом*, если $V' \subset V, E' \subset E$. Если граф G' является подграфом G , будем обозначать это $G' \subset G$.

Определение 5.3.2. Пусть $T\langle V_T, E_T \rangle \subset G\langle V, E \rangle$. Если T – дерево и более того $V_T = V$, тогда T – *остовное дерево* графа G . Остовное дерево также называют остовом.

Определение 5.3.3. Пусть $G\langle V, E \rangle$ – граф, $w : E \rightarrow \mathbb{R}$ – весовая функция. Тогда *весом графа* назовём следующую величину:

$$w(G) \stackrel{\text{def}}{=} \sum_{e \in E} w(e)$$

Определение 5.3.4. Пусть $G\langle V, E \rangle$ – связный граф, $w : E \rightarrow \mathbb{R}$ – весовая функция. *Минимальным остовным деревом* назовём такое остовное дерево, вес которого минимален среди всех других остовных деревьев графа G . Иногда минимальное остовное дерево называют *миностовом*.

Собственно задача, которую мы будем решать оставшуюся часть лекции – это алгоритмы построения таких миностовов. Все алгоритмы, которые мы разберём, по своей сути жадные и используют под капотом одну и ту же лемму, которую мы прямо сейчас докажем.

Далее под графом мы везде подразумеваем связный граф, если не оговорено обратное (ведь остовные деревья не в связных графах не имеют смысла).

5.3.1 Лемма о безопасном ребре

Примечание. Далее мы будем отождествлять множество рёбер E с соответствующим графом $G\langle V, E \rangle$, множество вершин которого совпадает с множеством вершин, используемых в рёбрах из E .

Всюду далее мы считаем, что у нас задан некоторый граф $G\langle V, E \rangle$ и весовая функция $w : E \rightarrow \mathbb{R}$.

Определение 5.3.5. Пусть A – некоторое множество рёбер, которое является подграфом некоторого миностова. Пусть $e \in E$. Ребро e будем называть *безопасным* относительно A , если $A \cup \{e\}$ – подграф некоторого миностова.

Ясно, что такие ребра чрезвычайно полезны и из определения безопасного ребра уже в принципе можно сделать алгоритмы, которые строят миностов. Но все это упирается в то, как искать вообще такие рёбра. Следующая лемма немного нам прояснит как они ведут себя.

Определение 5.3.6. Разрезом будем называть пару $(S, T) : S \cap T = \emptyset, S \cup T = V$.

Определение 5.3.7. Будем говорить, что ребро $(u, v) \in E$ пересекает разрез (S, T) , если $u \in S$ и $v \in T$, либо $u \in T$ и $v \in S$.

Лемма (О безопасном ребре) 5.3.1. Пусть множество рёбер A – подграф некоторого миностова. Пусть (S, T) – разрез G таков, что ни одно ребро из A не пересекает разрез (S, T) . Если $e \in E$ – ребро минимального веса из тех, что пересекают разрез (S, T) , то e – безопасное ребро для A .

Доказательство.

Пусть T – некоторый миностов, что $A \subset T$ (такой существует, по определению A). Если $A \cup \{e\} \subset T$, то теорема доказана. Рассмотрим случай, когда $A \cup \{e\} \not\subset T$. Будем считать, что $e = (u, v)$ и $u \in S, v \in T$.

Заметим, что при добавлении e в T образуется цикл (пример этого см. на рисунке ниже). Также заметим, что в этом цикле есть хотя бы одно ребро пересекающее разрез (S, T) отличное от (u, v) (на рисунке это ребро (b, c)). Обозначим данное ребро за e' .

Удалим ребро e' из T . Как следствие T разделяется на две компоненты связности. Теперь возьмем и соединим эти две компоненты ребром e , получив некоторое новое остовное дерево T' (на примере ниже, удаляем (b, c) и вставляем (u, v)).

Далее, заметим, что:

$$w(T') = w(T) - w(e') + w(e)$$

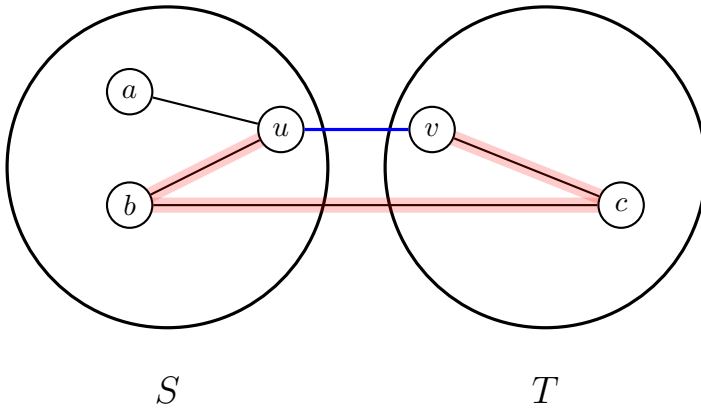


Рис. 5.2: Пример цикла, который образовался при добавлении ребра (u, v)

Также e' – ребро пересекающее разрез при том, что e выбрано как ребро минимального веса из всех пересекающих разрез. Из этого следует, что:

$$w(e) - w(e') \leq 0$$

Комбинируя с предыдущим равенством, получаем:

$$w(T') = w(T) - w(e') + w(e) \leq w(T)$$

$$w(T') \leq w(T)$$

Но в силу того, что T – остовное дерево минимального веса, то T' – тоже миностов. Теперь покажем, что $A \cup \{e\}$ есть подграф T' . Заметим, что $e' \notin A$, по определению разреза (S, T) . Поэтому если отождествить T и T' с множеством соответствующих рёбер, то получаем следующие соотношения:

$$A \subset T \setminus \{e'\} \Rightarrow A \cup \{e\} \subset (T \setminus \{e'\}) \cup \{e\} = T'$$

$$A \cup \{e\} \subset T'$$

Получили, что $A \cup \{e\}$ есть подграф некоторого миностова T' , значит, по определению, e – безопасное.

5.3.2 Алгоритм Крускала

Приведем первый алгоритм, который будет строить нам миностов – алгоритм Крускала.

Будем строить миностов поэтапно. Отсортируем рёбра по весу по возрастанию, а далее будем пытаться жадно строить из них дерево. Осталось только понять как строить дерево.

Изначально примем, что наш остов состоит только из вершин графа G без каких-либо рёбер. То есть получается каждая вершина на первом этапе представляет собой одну компоненту связности. По сути в начале у нас есть $|V|$ *непересекающихся множеств* $\{\{v_1\}, \{v_2\}, \dots, \{v_{|V|}\}\}$. Будем представлять их с помощью СНМ. Далее будем итерироваться по рёбрам, пусть мы рассматриваем ребро (v_i, v_j) . Если v_i и v_j находятся в разных компонентах связности, то мы можем добавить данное ребро в дерево и объединить соответствующие компоненты связности в СНМ. После итерации по всем рёбрам мы получим дерево.

Упражнение. Докажите, что на выходе мы действительно получим дерево.

Теперь представим код, выполняющий данный алгоритм. $\text{SORT_BY_W}(E, w)$ – сортирует рёбра по весу.

Algorithm 5 Алгоритм Крускала

```

1: function KRUSKAL( $G(V, E), w$ )
2:    $T \leftarrow \{\}$ 
3:   for  $v \in V$  do
4:     MAKE_SET( $v$ )
5:   SORT_BY_W( $E, w$ )
6:   for  $(u, v) \in E$  do
7:     if !SAME( $u, v$ ) then
8:        $T \leftarrow T \cup \{(u, v)\}$ 
9:       UNITE( $u, v$ )
10:  return  $T$ 

```

Утверждение 5.3.2. KRUSKAL работает за $\mathcal{O}(|E| \log |V|)$, если сортировка

работает за $\mathcal{O}(|E| \log |E|)$, а также СНМ используется с весовой эвристикой и эвристикой сжатия путей.

Доказательство.

Инициализация T работает за $\mathcal{O}(1)$, 3-4 строчки работают $\mathcal{O}(|V|)$. Далее сортировка работает за $\mathcal{O}(|E| \log |E|)$. А вот цикл на 6-ой строчке работает за $\mathcal{O}(|E|\alpha(|V|))$.

Значит итоговое время работы есть:

$$\begin{aligned} \mathcal{O}(|E| \log |E| + |E|\alpha(|V|)) &= \mathcal{O}(|E|(\log |E| + \alpha(|V|))) = \\ &= \mathcal{O}(|E|(\log |V|^2 + \alpha(|V|))) = \mathcal{O}(|E|(\log |V| + \alpha(|V|))) = \\ &= \{\alpha(|V|) = \mathcal{O}(\log |V|)\} = \mathcal{O}(|E| \log |V|) \end{aligned}$$

□

Утверждение 5.3.3. Результатом функции KRUSKAL является миностов.

Доказательство.

Утверждается, что после каждой итерации цикла в строчке 6 множество T является подграфом некоторого миностова. Покажем это по индукции.

База. $T = \emptyset$ – тривиально.

Пусть для каждого $0, 1 \dots n-1 < |V|$ доказали утверждение. Пусть мы рассматриваем на n -ом шаге ребро (u, v) . Если u и v лежат в одной компоненте связности (то есть $\text{SAME}(u, v)$ вернул истину), тогда утверждение тривиально, потому что мы ничего не поменяли. Пусть они лежали в разных компонентах. Для определённости, u и v лежат в компонентах A и B соответственно. Рассмотрим разрез $\mathcal{A} \stackrel{\text{def}}{=} (A, V \setminus A)$. Заметим, что ни одно ребро из T не пересекает $\mathcal{A}$. Более того, в силу отсортированности E по весам, ребро (u, v) в частности минимальное ребро, которое пересекает $\mathcal{A}$. Также вспомним, что, по предположению индукции, T – подграф некоторого миностова. Значит, по лемме о безопасном ребре, (u, v) – безопасное ребро для $T \Rightarrow T \cup \{(u, v)\}$ – подграф некоторого миностова. Доказали требуемое.

В силу того, что на каждом шаге T – подграф некоторого миностова, в

частности, и после последней итерации T – подграф некоторого миностова. Но после последней итерации T уже представляет собой дерево, а значит является миностовом.

□

5.3.3 Алгоритм Прима

Далее мы приведём похожий на алгоритм Дейкстры алгоритм Прима, который нам позволяет находить миностов.

Выберем начальную вершину r – корень нашего будущего миностова. Из этой вершины наш миностов будет «проростать». Далее все время будем жадно к нашему дереву присоединять вершины через минимальные рёбра и на выходе должен получиться навверное миностов (далее мы покажем, что это действительно так). Теперь к реализации. Будем поддерживать наше дерево неявно, как множество пар «вершина–родитель». Если мы построим алгоритм, что он будет нам возвращать такое множество, само дерево восстановить тривиально. Такое множество пар будем обозначать T . Оно задается следующим образом:

$$T \stackrel{def}{=} \{(\pi[v], v) \mid v \in V\}$$

Где π – отображение, которое вершине сопоставляет родителя в миностове.

Далее обстоит вопрос о том, как выбирать те самые минимальные рёбра. Будем поддерживать очередь вершин с приоритетами, где приоритет вершины v – это кратчайшее ребро до T (то есть до некоторой вершины T). Если такого ребра нет, то считаем, что это расстояние есть $+\infty$. Более формально положим:

$$k[v] \stackrel{def}{=} \min_{(u,v) \in T} w(u, v)$$

Обратите внимание, что k определяется относительно некоторой итерации алгоритма расширения остова.

Тогда результирующий алгоритм выглядит следующим образом (он очень похож и внешне и по механике на алгоритм Дейкстры):

Algorithm 6 Алгоритм Прима

```

1: function PRIM( $G\langle V, E \rangle, w$ )
2:    $k \leftarrow \{\}$ 
3:    $\pi \leftarrow \{\}$ 
4:   for  $v \in V$  do
5:      $k[v] \leftarrow +\infty$ 
6:      $\pi[v] \leftarrow \emptyset$ 
7:    $r \leftarrow V[0]$ 
8:    $k[r] \leftarrow 0$        $\triangleright$  Расстояние от корня до миностово равно нулю, что
      логично взять базой
9:    $Q \leftarrow \text{MAKE\_QUEUE}(V, k)$   $\triangleright$  Инициализируем приоритетную очередь
      на основе  $k$ 
10:  while  $Q \neq \emptyset$  do
11:     $v \leftarrow Q.\text{EXTRACT\_MIN}()$ 
12:    for  $(v, u) \in E$  do
13:      if  $u \in Q \wedge w(v, u) < k[u]$  then
14:         $k[u] \leftarrow w(v, u)$ 
15:         $\pi[u] \leftarrow v$                                  $\triangleright$  Обновляем родителя
16:         $Q.\text{DECREASE\_KEY}(u, w(v, u))$ 
17:  return  $\{(v, \pi[v]) \mid v \in V\}$ 

```

Утверждение 5.3.4. Функция PRIM возвращает миностов.

Доказательство.

Положим $\mathcal{V}_n$ – множество всех достанных вершин на строчке 11 после n итераций цикла. Тогда положим $\mathcal{T}_n \stackrel{\text{def}}{=} \{v \in \mathcal{V}_n \setminus \{r\} : (\pi[v], v)\}$ – граф построенный на n -ом шаге.

Покажем, что $\mathcal{T}_n$ на любом шаге является подграфом некоторого миностова по индукции.

База $n = 0$. $\mathcal{T}_0 = \emptyset$ – тривиально.

Пусть мы сделали $n_0 - 1$ шагов цикла ($n_0 \geq 1$). Покажем корректность для n_0 .

Пусть мы достали из Q вершину v (далее считаем, что в Q есть вершина

v). Рассмотрим разрез $(V \setminus Q, Q)$, если воспринимать Q как множество вершин. Тогда заметим, что $(\pi[v], v)$ пересекает разрез. Более того, легко видеть, что данное ребро минимально по весу среди всех остальных рёбер пересекающих данный разрез. Далее, по предположению индукции, $\mathcal{T}_{n_0-1}$ – подграф некоторого миностова. Значит данное ребро безопасно относительно $\mathcal{T}_{n_0-1}$, по лемме о безопасном ребре. Значит, $\mathcal{T}_{n_0} = \mathcal{T}_{n_0-1} \cup \{(\pi[v], v)\}$ – подграф некоторого миностова. Получили требуемое.

□

Далее встаёт вопрос об асимптотике, которая напрямую зависит от реализации приоритетной очереди.

Утверждение 5.3.5. Если использовать бинарную пирамиду в качестве приоритетной очереди, то время работы есть $\mathcal{O}(|E| \log |V|)$

Доказательство.

Построение кучи занимает $\mathcal{O}(|V|)$. Далее **while** делает **EXTRACT_MIN** $\mathcal{O}(|V|)$, значит они суммарно работают за $\mathcal{O}(|V| \log |V|)$. Далее, для каждого ребра (заметим, что каждое ребро просматривается ровно один раз) в худшем случае мы будем обновлять кучу за $\mathcal{O}(\log |V|)$. Значит, суммарно **DECREASE_KEY** работает за $\mathcal{O}(|E| \log |V|)$.

Итого, результирующее время выполнения $\mathcal{O}(|V| \log |V| + |E| \log |V|) = \mathcal{O}(|E| \log |V|)$.

□

Казалось бы, получилось такая же временная сложность как и у алгоритма Крускала, но в отличие от него, алгоритм Прима можно ускорить. Следующее утверждение не будет спрашиваться на экзамене.

Утверждение 5.3.6 (б/д). Если использовать фибоначчиеву кучу, то время работы $\mathcal{O}(|E| + |V| \log |V|)$.

Доказательство.

Проводя аналогичные рассуждения, только учтя, что **EXTRACT_MIN** работает за $\mathcal{O}^*(\log |V|)$, а **DECREASE_KEY** за $\mathcal{O}^*(1)$. Получаем, что время выполнения $\mathcal{O}(|V| \log |V| + |E| \cdot 1) = \mathcal{O}(|V| \log |V| + |E|)$.

